Testing

- Il software testing
- White & Black box testing
- Livelli di testing
- Planning test

Presented with xmind

White & Black box testing

Complementarietà dei due approcci

- Non si può stabilire quale approccio sia il migliore in senso assoluto
  - Dipende dal contesto
- Studio suggerisce che il testing funzionale rivela più difetti con tester esperti rispetto al testing strutturale

Black Box Testing

- Non si basa sulla conoscenza del SUT e della sua struttura interna
- Anche chiamato functional testing
- Basato sui requisiti o specifiche
- Può essere Attività di Verifica o Validazione
- Tester non ha accesso al codice, lo vede come una black box
- Obiettivo: garantire che il modulo o il sistema sia in accordo con le specifiche
- Utilizza tecniche di Test Funzionali e Test Comportamentali
- Individua 5 tipi di errori:
  - **Funzioni mancanti o non corrette**
  - **Errori nell'interfaccia**
  - **Errori nelle strutture dati delle interfacce**
  - **Errori di prestazioni e comportamento**
  - **Errori di inizializzazione e terminazione**
- Vantaggi:
  - Utilizzabile su qualsiasi tipo di sistema
  - Può essere utilizzato per eseguire il System Testing
  - Casi di test possono essere scritti prima del codice
  - Spesso affidato a gruppi di testing indipendenti o terze parti

White Box Testing

- Conoscenza esplicita del System Under Test (SUT) e della sua struttura
- Anche chiamato structural testing
- Basato sul concetto di "copertura"
  - l'obiettivo è quello di provare tutti i cammini di esecuzione presenti nel software.
- Esempio: Statement coverage testing
  - Attività di verifica dove il tester ha accesso al codice e spesso coincide con l'autore del codice
- Esamina logica e struttura del codice
- Unit test utilizzano tecniche di white box testing
  - Una unit è un componente del codice
  - Permettono di catturare il 65% dei bug
- Obiettivi:
  - garantire che tutti i percorsi indipendenti di un modulo siano eseguiti almeno una volta;
  - Eseguire tutte le decisioni logiche
  - Eseguire tutti i loop compresi gli estremi
  - Check delle strutture interne di dati
- Limiti:
  - Numero di casi di test può diventare molto grande
  - Utilizzato principalmente per Unit Testing e porzioni del sistema
    - Testing in the small
  - Non rivela "missing feature errors"
    - Che possono essere scoperti solo considerando i requisiti/specifiche

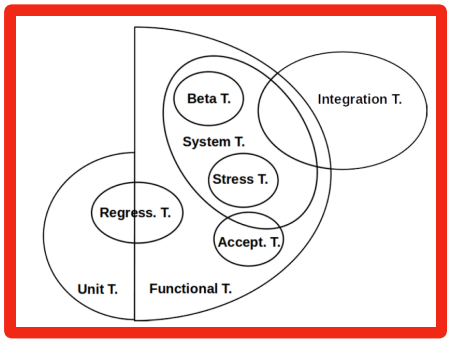
Presented with xmind

Il software testing

- È una procedura sistematica che prevede l'esecuzione di un sistema software con l'intento di trovare difetti, noti come faults
  - **System Under Test (SUT)**
- Secondo la definizione IEEE, è un processo di analisi del prodotto software al fine di eliminare eventuali discrepanze tra le condizioni esistenti e quelle richieste (ovvero, gli errori) e per valutare le caratteristiche del software
- Per i risultati corretti e non corretti, ci sono diverse definizioni da considerare
  - **Mistake (errore umano)**
    - Si riferisce ad un'azione umana che porta ad un risultato non corretto
  - **Fault or Defect (Difetto)**
    - Si riferisce ad un passaggio, processo o definizione di dati non corretti all'interno del programma
  - **Failure (Fallimento)**
    - Si riferisce all'incapacità del sistema o di un suo componente di svolgere una funzione richiesta entro specifiche prestazioni desiderate
  - **Error (errore)**
    - Si riferisce alla differenza tra un valore o condizione calcolati, osservati o misurati e un valore o condizione reali, desiderati e teoricamente corretti
- Il testing è correlato alla Verifica e Validazione (V&V) del software
  - La Verifica è il processo di valutazione di un sistema o componente per determinare se il prodotto di una specifica fase di sviluppo soddisfa le condizioni imposte all'inizio di tale fase (IEEE 1990)
    - Assicura che il software implementi in modo corretto le specifiche funzioni, rispettando i requisiti imposti
    - Permette di rispondere alla domanda: stiamo implementando correttamente il prodotto?
  - La Validazione è il processo di valutazione di un sistema o componente durante o alla fine del processo di sviluppo per determinare se soddisfa specifici requisiti (IEEE 1990)
    - Si cerca di accertare se il prodotto in sviluppo sia realmente quello giusto e se soddisfi le richieste del cliente.
    - La validazione ha a che fare con la correttezza delle specifiche rispetto alle reali esigenze del cliente
- Test Case
  - Sequenza di passi per testare una funzionalità o aspetto
  - Accerta la correttezza del comportamento
  - Include descrizione degli input e del comportamento atteso
  - Serie di input, precondizioni e risultati attesi
  - Chiamato anche test data
  - **Una collezione di Test case è chiamata Test suite.**
- Testing Esaustivo
  - Approccio di testing che testa con tutti i possibili input
  - **Input domain può diventare molto grande**
  - Testing esaustivo diventa impraticabile
  - Necessarie strategie di input selection
- Processo di Sviluppo del Software
  - Test eseguiti il prima possibile e più volte possibile
    - Massimizza la probabilità di trovare problemi
  - **Test ripetibili, riusabili e possibilmente automatici**
    - Eseguiti come codice
    - Scritti con i requisiti e aggiornati con i cambiamenti
    - Garantiscono una buona qualità del prodotto
- Assicura il comportamento previsto dell'applicazione
  - Riduce il costo totale di manutenzione per gli utenti finali
  - Aumenta la soddisfazione degli utenti
- Riduce i costi di sviluppo
  - Problemi non riscontrati in produzione costano di più
    - da 40 a 100 volte
  - Risoluzione durante le fasi di sviluppo è più economica
- I principali obiettivi del Software Testing sono
  - la verifica della qualità generale del software;
  - la scoperta dei "bug" o difetti nelle applicazioni;
  - la verifica della rispondenza alle specifiche, e la congruità delle stesse;
  - dell'integrazione tra i sistemi;
  - delle modifiche apportate avendo cos` ı l'evidenza di eventuali effetti non voluti, attraverso i cosiddetti test di regressione.

Presented with xmind

Livelli di testing



Stress Testing

- Test intensi e approfonditi per valutare la stabilità
- Test oltre la capacità operativa normale
- Verifica robustezza, disponibilità, gestione errori
- Test fino al punto di rottura

Beta Testing

- Offerta gratuita di una versione avanzata del software a beta tester
- Raccolta di segnalazioni di difetti e feedback dagli utenti
- Test di system testing effettuato utilizzando realmente il sistema
- Test black-box

Regression Testing

- Ripetizione selettiva del test per verificare che le modifiche apportate non abbiano causato effetti indesiderati
- Assicura che il sistema soddisfi ancora i requisiti
- verifica che non vi siano stati side effect dopo una modifica
- Utilizzo di suite di test automatici
- Test black-box e white-box

Acceptance Testing

- Determina se il sistema soddisfa i criteri di accettazione del cliente
- Il cliente valuta l'accettabilità del sistema
- Utilizzo specifico di Test funzionali o di system testing
- Test black-box

Unit Testing

- Testa singole unità (funzioni, metodi, classi)
- Utilizza tecniche white-box
- Lo sviluppatore isola l'unità dal resto del sistema
- Utilizzo di oggetti fittizi (stub/mock object) che simulano il contesto reale del test.

Integration Testing

- Testa l'interazione dei componenti software/hardware
- Verifica la comunicazione e lo scambio di informazioni tra moduli
- Errori comuni nelle interfacce tra moduli
- Utilizza tecniche white-box e black-box

Functional Testing

- Verifica che le funzionalità rispettino i requisiti specificati
- Basato sulle specifiche dei requisiti del software
- Utilizza tecniche Black-box

System Testing

- Testa un sistema completo e integrato
- Verifica la conformità alle specifiche
- Valuta correttezza, prestazioni, usabilità, sicurezza
- Test black-box

Presented with xmind

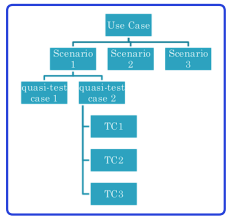
Planning test

Principio "Test Early and Often"

- Esecuzione dei test case il prima possibile per ricevere feedback tempestivi
- È importante eseguire i test fin dalle prime fasi dello sviluppo
- **Integrazione Continua**
  - È necessario integrare i vari pezzi di codice in un unico codice del sistema completo
  - È raccomandato effettuare l'integrazione almeno una volta al giorno
- **Esecuzione dei Test Automatizzati**
  - Dopo l'integrazione, eseguire tutti i test automatizzati
  - Verifica che il sistema software funzioni correttamente e senza errori
- **Benefici del Test Early and Often**
  - Garanzia di qualità del codice
  - Ricezione di feedback tempestivi sulle funzionalità implementate
  - Rilevazione precoce di eventuali problemi o bug

Approccio Three Step per la Generazione dei Test Case

- Generazione degli scenari dai casi d'uso
  - Creazione di un insieme completo di scenari per ogni caso d'uso
- Identificazione delle condizioni di esecuzione (Quasi-Test Case)
  - Identificazione delle condizioni necessarie per eseguire uno scenario
- Creazione dei casi di test
  - Identificazione delle combinazioni di input per il quasi-test case
  - Creazione di uno o più casi di test basati su queste combinazioni



Piano di Test/Test Plan

- È un documento che descrive
  - Obiettivo del piano di test
  - Approccio al testing
  - Risorse necessarie per il testing
  - Pianificazione delle attività di test
- Individua
  - Elementi e aspetti da testare
  - Attività di test
  - Eventuali piani di emergenza per gestire rischi imprevisti
- Definizione dei casi di test è fondamentale

IEEE 1990

Formato del Test Case

- **Test Id** — Identificatore del test case
- **Description** — Descrizione dei passi da eseguire e delle condizioni di ingresso
- **Expected Results** — Risultati attesi
- **Actual Results** — Risultati ottenuti dal test

Test ID	Description	Expected Results	Actual Results

Fattori importanti dei Test Case

- Chiarezza e non ambiguità dei campi del test case
- Riproducibilità dei problemi riscontrati
- Corrispondenza ai requisiti del cliente
- Dovrebbe essere definito il prima possibile nel ciclo di sviluppo

Presented with xmind